

BÁO CÁO BÀI THỰC HÀNH: XÂY DỰNG CHƯƠNG TRÌNH CHƠI CỜ CARO AI

Sinh viên thực hiện: Nguyễn Đức Anh - Nguyễn Trọng Huy
Mã số sinh viên: 22022661 - 22022545

1. GIỚI THIỆU VỀ BÀI TOÁN

1.1. Mô tả bài toán cờ Caro

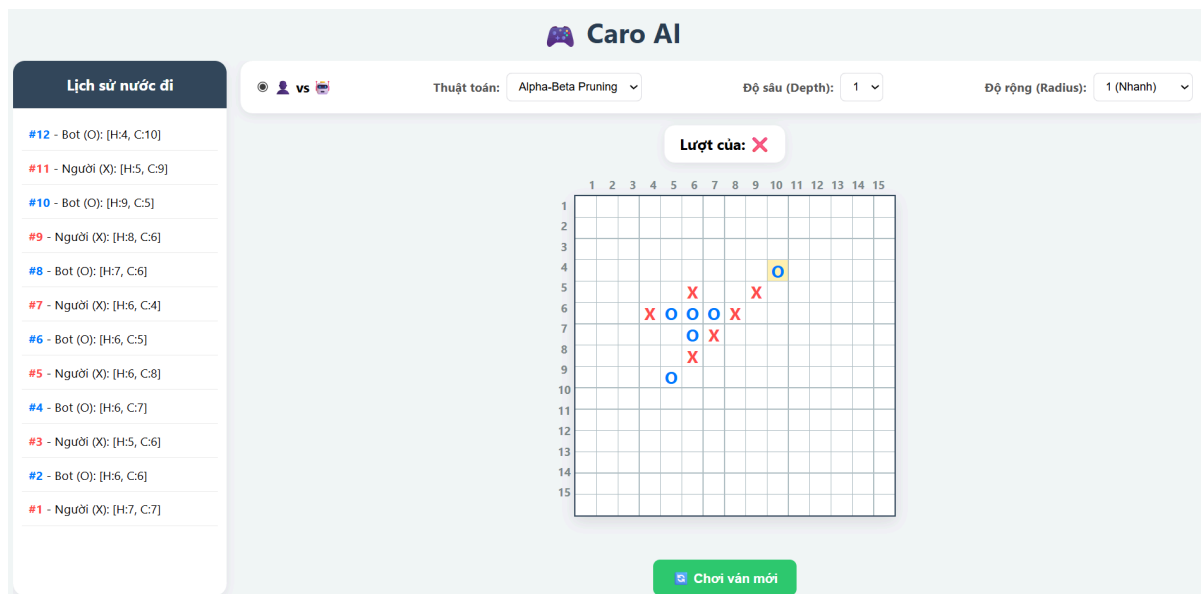
- Kích thước bàn cờ: 15x15
- Đối tượng tham gia: Người chơi (X) và Máy tính (O).
- Mục tiêu: Xây dựng trò chơi giữa người với máy sao cho máy có thể tự động đánh vào nước đi hợp lý với 3 mức độ ngẫu nhiên, ứng dụng thuật toán minimax với độ sâu hợp lý và sử dụng cắt tia alpha beta để tối ưu cho thuật toán minimax khi tăng độ sâu.

1.2. Luật chơi và điều kiện thắng

- Luật đi quân: Hai bên luân phiên đặt quân vào ô trống tương ứng với ký hiệu của mình X với người chơi và O với máy.
- Điều kiện thắng: Có 5 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.
- Luật bổ sung: Không áp dụng luật chặn hai đầu.
- Điều kiện hòa: Bàn cờ đầy.

1.3. Giao diện người chơi

- Giao diện web đơn giản gồm một bàn cờ với kích thước 15x15 ô, có hiển thị số thứ tự ở 2 biên giúp người chơi nhận diện nước đi dễ hơn
- Thiết lập thuật toán, độ sâu, độ rộng của máy.
- Hiển thị số nước đi tương ứng của người và máy.
- Giao diện mượt mà, các logic đảm bảo trong quá trình máy đang tính toán thì người không thể đi nước tiếp.



2. PHƯƠNG PHÁP CÀI ĐẶT KỸ THUẬT

2.1. Cách biểu diễn trạng thái bàn cờ

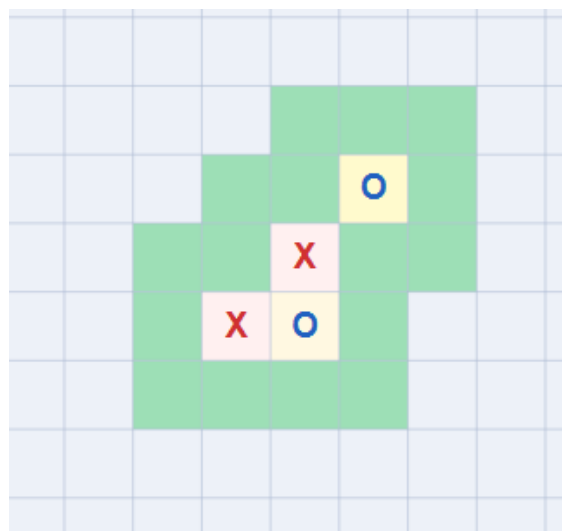
- Cấu trúc dữ liệu: sử dụng cấu trúc mảng hai chiều trong python để biểu diễn trạng thái của bàn cờ: `board[list[list[str]]]`.
- Cách biểu diễn quân cờ: sử dụng kiểu dữ liệu string trong python để biểu diễn “X” là người chơi, “O” là máy và “.” là ô trống chưa được đánh.
- Biểu diễn nước đi: một nước đi là một cặp hai số nguyên (r, c) tương ứng: r: số thứ tự của hàng, c số thứ tự của cột. Nước đi hợp lệ phải thỏa mãn thỏa mãn r, c nhỏ hơn kích thước bàn cờ và lớn hơn 0, đồng thời giá trị tại bàn cờ tại hàng cột đó phải là trống.
- Hàm chuyển trạng thái: `make_move`
 - Input: nước đi hợp lệ và ký hiệu của người hoặc máy tùy theo lượt.
 - Output: thay đổi giá trị của bàn cờ theo nước đi đó và ký hiệu X hoặc O tương ứng.
- Hàm quay lại trạng thái cũ: `undo_move`
 - Input: nước đi hợp lệ đã được đi trước đó và ký hiệu của người hoặc máy tùy theo lượt.
 - Output: thay đổi giá trị của bàn cờ theo nước đi đó thành trống.

2.2. Cách sinh nước đi tiềm năng.

Nước đi tiềm năng: là những nước đi có khả năng cao sẽ được đi vào nhất thay vì phải xét toàn bộ nước đi hợp lệ. Trong hệ thống sử dụng cơ chế sinh nước đi tiềm năng dựa trạng thái bàn cờ hiện tại.

Cụ thể: hệ thống sẽ duyệt qua toàn bộ các hàng và cột trong bàn cờ hiện tại và đánh dấu những nước cờ đã được người chơi hoặc máy đánh vào. Tại mỗi nước cờ đó mở rộng tất cả các nước cờ hợp lệ (ở trạng thái trống) xung quanh bao gồm trên dưới trái phải và 4 nước chéo và thêm vào mảng tập hợp các nước cờ tiềm năng.

Thêm vào đó qua giao diện người dùng có thể cấu hình được bán kính mở rộng với mỗi nước cờ đã đi. Mặc định hệ thống sử dụng $r = 1$: mở rộng xung quanh trong phạm vi 1 ô. Tuy nhiên để tránh thời gian chạy quá tải người dùng chỉ được mở rộng với r tối đa bằng 3.



Trong hình trên các ô màu xanh lá được coi là các ô tiềm năng và trong quá trình tính toán thuật toán thì hệ thống chỉ xét các nước đi tiềm năng thay vì xét toàn bộ nước đi hợp lý.

- Hàm lấy nước đi tiềm năng: *get_potential_moves*
 - Input: trạng thái hiện tại của bàn cờ, bán kính mở rộng r
 - Output: tập hợp các nước đi tiềm năng xung quanh những nước cờ đã đi với bán kính r .

Cải tiến: Trong thực tế khi số nước đi đã đánh trong bàn cờ tăng cao thì ngay cả việc xét các nước đi với bán kính $r=1$ cũng trở nên rất lớn do đó. Hệ thống cải thiện bằng cách tính toán nhanh giá trị trạng thái bằng hàm đánh giá nước đi *score_move()*. Sau đó sẽ sắp xếp lại tập các nước đi tiềm năng theo số điểm tương ứng. Điều này giúp thuật toán alpha-beta có thể cắt tỉa nhanh hơn gặp những nước đi tốt nhất từ đầu. Đồng thời khi số lượng nước đi tiềm năng lớn, sau khi sắp xếp hệ thống sẽ chọn ra N nước đi đầu tiên với số điểm cao nhất giảm việc phải xét toàn bộ các nước đi tiềm năng nhưng với số điểm thấp.

2.3. Cách kiểm tra trạng thái kết thúc

- Trạng thái kết thúc của một ván cờ sẽ là khi người hoặc máy giành chiến thắng hoặc kết quả là hòa trong trường hợp không còn nước đi hợp lệ: bàn cờ bị lấp đầy.
- Kiểm tra chiến thắng: hệ thống sẽ dựa trên trạng thái của bàn cờ khi có liên tiếp 5 ký tự X hoặc O cùng xếp thành một đường chéo, thẳng hàng hoặc một cột mà không có ký tự nào khác xen giữa. Cụ thể tại mỗi ô trong bàn cờ đã được đánh hệ thống sẽ xem xét lần lượt:
 - 4 ô theo hàng ngang: *all(board[r][c+i] == player for i in range(5))*
 - 4 ô theo hàng dọc: *all(board[r+i][c] == player for i in range(5))*
 - 4 ô theo đường chéo chính (\): *all(board[r+i][c+i] == player for i in range(5))*
 - 4 ô theo đường chéo phụ (/): *all(board[r+i][c-i] == player for i in range(5))*

Nếu có bất kỳ bộ 4 ô nào thỏa mãn (cộng với ô đang xét là 5) thì người chơi mang ký hiệu tương ứng sẽ giành chiến thắng. Không tính luật chặn hai đầu

2.4. Các hàm tính toán giá trị nước đi.

- Đây là các hàm giúp hỗ trợ việc sắp xếp các nước đi tiềm năng đảm bảo các nước đi có giá trị cao được đẩy lên đầu giúp giảm thời gian tính toán trong các thuật toán alpha-beta.
- Hàm *score_move()*: được sử dụng để đánh giá mức độ tiềm năng của một nước. Hàm hoạt động bằng cách kiểm tra nước đi theo bốn hướng chính gồm ngang, dọc và hai đường chéo. Với mỗi hướng, hàm sẽ quét tối đa 4 ô về cả hai phía để đếm số quân cờ liên tiếp của AI và đối thủ, từ đó đánh giá đồng thời khả năng tấn công và phòng thủ. Các chuỗi quân cờ càng dài sẽ được gán trọng số điểm càng cao, ví dụ chuỗi có khả năng tạo 5 quân liên tiếp sẽ nhận điểm rất lớn. Ngoài ra, hàm còn ưu tiên các nước đi gần trung tâm bàn cờ nhằm giúp AI mở rộng thế trận hiệu quả hơn. Kết quả cuối cùng là một điểm số đại diện cho mức độ quan trọng của nước đi, giúp thuật toán ưu tiên xét các nước đi tiềm năng trước để tăng hiệu quả tìm kiếm.
- Hàm *quick_score()*: là phiên bản đánh giá nhanh, tối ưu cho việc sắp xếp nước đi ban đầu. Thay vì quét sâu và phân tích nhiều trường hợp, hàm chỉ đếm số quân liên tiếp

của AI và đối thủ xung quanh vị trí đang xét. Sau đó gán trọng số cho các mẫu như chặn 4 hoặc tạo 4. Hàm này đơn giản và tốc độ nhanh hơn nhưng độ chính xác chiến thuật thấp hơn *score_move()*.

2.5. Cơ chế Zobrist Hashing

Zobrist Hashing là cơ chế tạo mã hash cho trạng thái bàn cờ bằng cách gán một số ngẫu nhiên 64-bit cho mỗi ô và từng loại quân cờ. Khi có nước đi mới, giá trị hash được cập nhật nhanh bằng phép XOR mà không cần duyệt lại toàn bộ bàn cờ. Cơ chế này giúp nhận diện nhanh các trạng thái đã tính toán trước đó để lưu và tra cứu trong Transposition Table, từ đó tăng tốc thuật toán tìm kiếm.

Zobrist Hashing — *get_board_hash()*

Biến bàn cờ 2D thành 1 số nguyên 64-bit duy nhất bằng phép XOR. Dùng để tra cứu Transposition Table trong $O(1)$.

2.6. Hàm đánh giá trạng thái

Hàm đánh giá trạng thái là một hàm heuristic dùng để đánh giá giá trị của một trạng thái hiện tại. Cụ thể với mỗi trạng thái bàn cờ hệ thống sẽ quét qua bộ 5 nhóm ô liên tiếp theo (window):

- Hàng ngang: $window = board[r][c:c+5]$
- Hàng dọc: $window = [board[r+i][c] \text{ for } i \text{ in range}(5)]$
- Đường chéo chính: $window = [board[r+i][c+i] \text{ for } i \text{ in range}(5)]$
- Đường chéo phụ: $window = [board[r+i][c-i] \text{ for } i \text{ in range}(5)]$

Và bỏ qua bộ 5 ô rỗng. Sau đó với mỗi 5 nhóm ô hệ thống sẽ tính toán điểm cho từng nhóm 5 ô đó (window). Cụ thể trong hệ thống được cài đặt cơ chế tính điểm như sau:

Đều số quân cờ X (người chơi) và O (máy chơi) và ô rỗng trong số 5 ô đó:

Nếu Máy có càng nhiều ô cộng càng nhiều điểm nhằm đẩy mạnh khả năng tấn công

Nếu Người có càng nhiều ô phải trừ nhiều điểm tương ứng để ép Máy phải phòng thủ

Điểm số cuối cùng của trạng thái sẽ là tổng điểm của tất cả các nhóm 5 ô theo cả 4 hướng ngang dọc chéo chính / phụ. Hiện tại hệ thống chỉ đếm số lần xuất hiện và bỏ qua vị trí của các ô trống trong window.

Bảng thông số điểm cộng cho từng điều kiện trong window

Điều kiện	Điểm số	Ý nghĩa
<code>bot_count == 5</code>	+ 2,000,000	Máy thắng ngay lập tức
<code>bot_count == 4</code> và <code>empty_count == 1</code>	+ 50,000	Máy sắp thắng, ưu tiên tấn công mạnh
<code>bot_count == 3</code> và <code>empty_count == 2</code>	+ 5,000	Tạo thế tấn công tiềm năng
<code>bot_count == 2</code> và <code>empty_count == 3</code>	+ 500	Thế tấn công yếu, tạo nền phát triển
<code>human_count == 4</code> và <code>empty_count == 1</code>	- 1,000,000	Người chơi sắp thắng, bắt buộc phải chặn

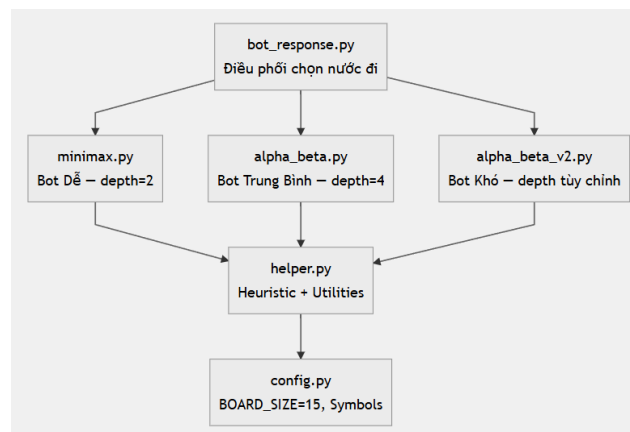
human_count == 3 và empty_count == 2	- 20,000	Người chơi có thể nguy hiểm
human_count == 2 và empty_count == 3	- 1,000	Người chơi đang tạo thế ban đầu

- Hàm `evaluate_board`
 - Input: trạng thái của bàn cờ board
 - Output: giá trị trạng thái đó
- Hàm `evaluate_window`:
 - Input: nhóm 5 ô liên tiếp window.
 - Output: điểm cho nhóm 5 ô đó.

2.7 Kiến trúc hệ thống

Backend: FastAPI nhận request gửi về từ phía frontend gồm có trạng thái bàn cờ board, thuật toán sử dụng, độ sâu depth và độ rộng radius. Sau đó sẽ gọi tới các hàm lấy nước đi tương ứng với các tham số trên.

Frontend: sử dụng website tĩnh với HTML + CSS + Javascript. Hiển thị trạng thái bàn cờ và gửi request lên FastAPI lấy nước đi tiếp theo.



Luồng xử lý chung: request → `bot_response.py` nhận bàn cờ → gọi thuật toán tương ứng → thuật toán đệ quy duyệt cây trạng thái → gọi `evaluate_board()` tại lá → trả về nước đi tốt nhất.

Module `bot_respose`, cung cấp các hàm tra về nước đi tương ứng với thuật toán lựa chọn, độ sâu và độ rộng được lựa chọn.

Cụ thể mỗi hàm `choose best move_by_[alpha_beta, minimax, alpha_beta_v2]` đều có chung cơ chế, gọi hàm lấy nước đi tiềm năng, gọi thuật toán tương ứng cho mỗi nước đi, trả về nước đi có số điểm cao nhất. Đối với các thuật toán alpha beta thì có thêm phần khởi tạo alpha, beta là số âm, dương vô cùng lần lượt. Bên cạnh đó kèm theo log ghi lại quá trình xử lý của từng thuật toán như: số trạng thái đã xét, thời gian xử lý, số nước được tận dụng cơ chế cache, số lần cắt tỉa.

```
def choose_best_move_by_alpha_beta(board, depth=4, radius=1):
    """
    Hàm chọn nước đi tốt nhất sử dụng thuật toán alpha_beta:
    Inout: board: trạng thái hiện tại của bàn cờ, depth: độ sâu duyệt, radius: bán kính các nước đi tiềm năng
    Output: best_move: cặp row, col nước đi tốt nhất trong bàn cờ hiện tại
    """
    start_time = time.time()
    stats = {
        "nodes": 0, # khởi tạo số trạng thái chưa xét
        "cutoffs": 0 # Số trạng thái bị cắt
    }
    best_move = None # nước đi tốt nhất
    best_score = -float('inf') # số điểm của nước đi

    alpha = -float('inf') # khởi tạo alpha
    beta = float('inf') # khởi tạo beta

    moves = get_ordered_moves(board, radius=radius)

    for move in moves:
        make_move(board, move, BOT_SYMBOL) # giả lập thực hiện nước đi
        score = alphabeta(board, depth - 1, alpha, beta, False, radius, stats) # tính toán giá trị của nước đi đó
        undo_move(board, move) # quay lại nước đi đó
        if score > best_score: # lưu dấu nước đi tốt nhất
            best_score = score
            best_move = move

        alpha = max(alpha, best_score)
    end_time = time.time()
    duration = end_time - start_time # ước tính thời gian
    print(f"\n--- PHÂN TÍCH ALPHA-BETA ---")
    print(f"Độ sâu {depth} độ rộng {radius}")
    print(f"⌚ Thời gian: {duration:.4f} giây")
    print(f"🔍 Số nút đã duyệt: {stats['nodes']}")
    print(f"🗑️ Số lần cắt tia: {stats['cutoffs']} lần")
    print(f"-----\n")
    return best_move
```

Hình minh họa hàm `choose_best_move` bằng thuật toán `alpha_beta`

```
--- PHÂN TÍCH ALPHA-BETA V2 ---
Độ sâu 4 độ rộng 1
⌚ Thời gian: 3.7268 giây
🔍 Số nút đã duyệt: 4006
🗑️ Tận dụng Cache: 665 lần
🗑️ Số lần cắt tia: 705 lần
-----
```

Hình minh họa log ghi lại quá trình chạy để tìm ra nước đi tối ưu

3. THUẬT TOÁN AI

3.1. Thuật toán Minimax (Level 1)

Thuật toán Minimax là một phương pháp tìm kiếm và ra quyết định thường được sử dụng trong các trò chơi đối kháng như cờ caro, cờ vua hay tic-tac-toe. Thuật toán hoạt động dựa trên việc giả lập tất cả các nước đi có thể xảy ra trong tương lai. Người chơi AI sẽ chọn nước đi mang lại giá trị tốt nhất cho mình (maximize), đồng thời giả định đối thủ luôn chơi tối ưu để làm giảm lợi thế của AI (minimize). Bằng cách đánh giá các trạng thái bàn cờ thông qua hàm lượng giá, Minimax giúp AI tìm ra nước đi tối ưu nhất trong từng tình huống.

- Maximizer (Bot): Chọn nước đi có điểm cao nhất.
- Minimizer (Người): Chọn nước đi có điểm thấp nhất (bất lợi cho Bot).

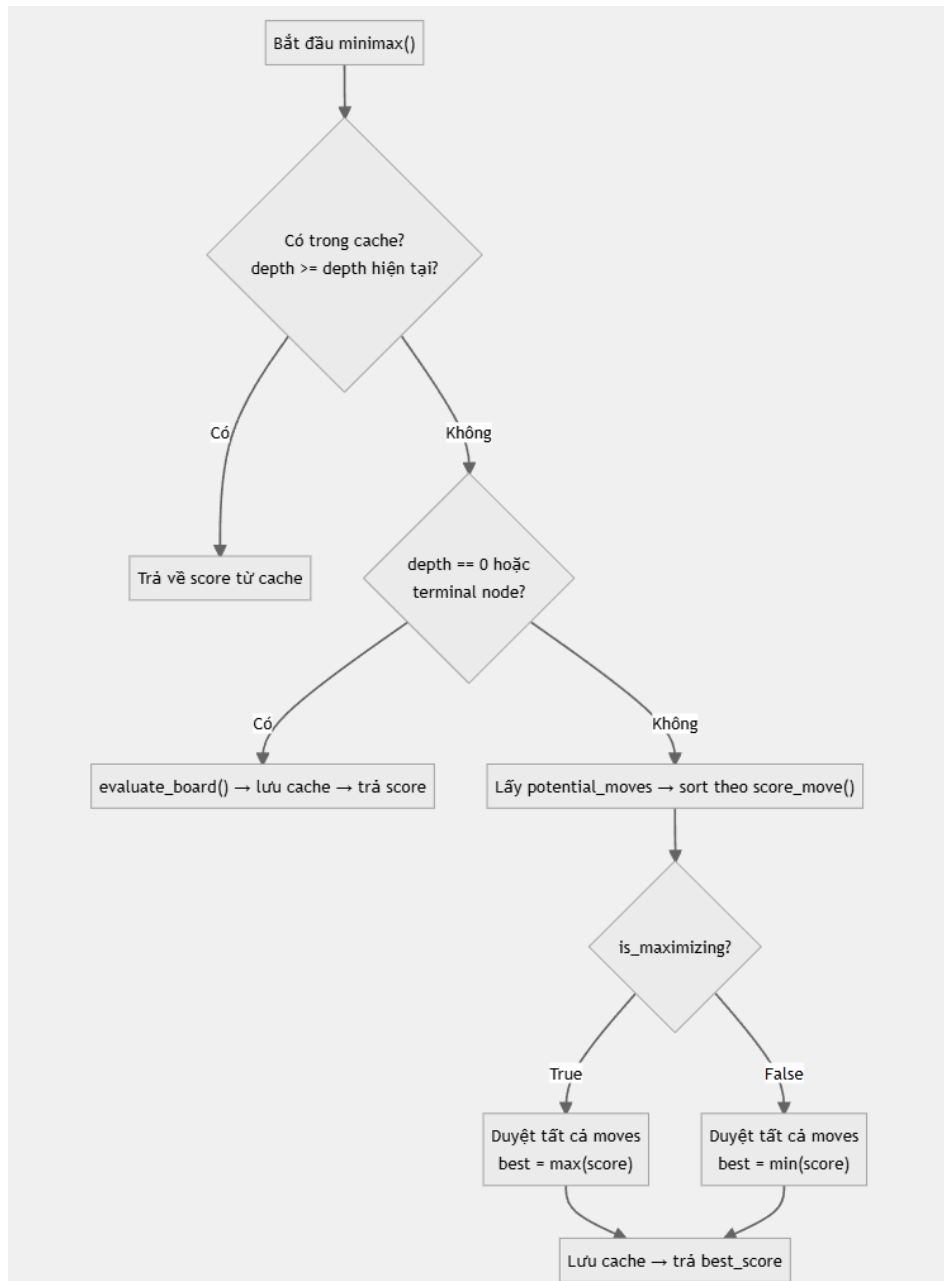
Thuật toán duyệt toàn bộ cây trạng thái đến độ sâu cho trước, không có cơ chế cắt tia nhánh. Trong hệ thống thuật toán được triển khai bằng cách Thuật toán hoạt động bằng cách đệ quy duyệt qua các nước đi tiềm năng, trong đó AI sẽ cố gắng chọn nước đi có điểm số cao nhất (maximizing), còn người chơi sẽ chọn nước đi làm giảm điểm số của AI (minimizing). Ở mỗi trạng thái bàn cờ, hàm `evaluate_board()` được sử dụng để đánh giá mức độ lợi thế của AI.

Ngoài ra trong hệ thống để tối ưu tốc độ xử lý, thuật toán sử dụng Transposition Table nhằm lưu lại các trạng thái bàn cờ đã được tính trước đó, giúp tránh việc đánh giá lặp lại cùng một trạng thái. Ngoài ra, danh sách nước đi được sắp xếp theo mức độ tiềm năng để ưu tiên xét dựa trên score_move các nước đi tốt trước, góp phần giảm độ phức tạp của cây tìm kiếm và tăng hiệu quả khi xử lý các bàn cờ lớn như cờ caro.

Mã giả thuật toán minimax được triển khai trong hệ thống:

```
def MINIMAX(board, depth, is_maximizing, radius):
    board_hash ← get_board_hash(board)
    NẾU board_hash có trong TRANSPOSITION_TABLE_MINIMAX
        TRẢ VỀ score đã lưu
    NẾU depth = 0 hoặc is_terminal_node(board)
        score ← evaluate_board(board)
        Lưu score vào TRANSPOSITION_TABLE_MINIMAX
        TRẢ VỀ score
    potential_moves ← get_potential_moves(board, radius)
    NẾU is_maximizing
        best_score ← -inf
        CHO MỖI move TRONG potential_moves
            make_move(board, move, BOT_SYMBOL)
            score ← MINIMAX(board, depth - 1, FALSE, radius)
            undo_move(board, move)
            best_score ← MAX(best_score, score)
    NGƯỢC LẠI
        best_score ← +inf
        CHO MỖI move TRONG potential_moves
            make_move(board, move, HUMAN_SYMBOL)
            score ← MINIMAX(board, depth - 1, TRUE, radius)
            undo_move(board, move)
            best_score ← MIN(best_score, score)
    Lưu best_score vào TRANSPOSITION_TABLE_MINIMAX
    TRẢ VỀ best_score
```

Luồng thực thi:



Các tối ưu đã áp dụng:

Kỹ thuật	Chi tiết
Transposition Table	TRANSPPOSITION_TABLE_MINIMAX — lưu {hash: {depth, score}}, tránh tính lại trạng thái trùng
Move Ordering	Sort theo score_move() giảm dần — nước tốt nhất xét trước
Beam Search	Dòng potential_moves[:10] đã bị comment → hiện duyệt tất cả nước đi

Giới hạn và hiệu quả:

Thuộc tính	Giá trị
------------	---------

Depth mặc định	2(trong <i>choose_best_move_by_minimax</i>)
Branching factor	Không giới hạn (beam search đã bị comment)
Độ phức tạp	$O(b^d)$ — với b = số nước đi, d = depth
Cắt tia	✗ Không có — duyệt toàn bộ nhánh

3.2. Thuật toán Alpha-Beta Pruning

Thuật toán Alpha-Beta là phiên bản tối ưu của Minimax, được sử dụng để giảm số lượng trạng thái cần duyệt trong cây tìm kiếm. Thuật toán hoạt động tương tự Minimax khi AI cố gắng tối đa hóa điểm số còn đối thủ cố gắng tối thiểu hóa điểm số của AI. Tuy nhiên, Alpha-Beta sử dụng hai giá trị alpha và beta để cắt tia những nhánh không cần thiết. Alpha đại diện cho giá trị tốt nhất mà AI đã tìm được, còn beta là giá trị tốt nhất của đối thủ. Khi phát hiện một nhánh chắc chắn không thể tạo ra kết quả tốt hơn nhánh đã xét trước đó, thuật toán sẽ dừng việc duyệt nhánh đó. Nhờ kỹ thuật cắt tia này, Alpha-Beta giúp tăng tốc độ xử lý đáng kể nhưng vẫn đảm bảo tìm được nước đi tối ưu như Minimax.

- Alpha (α): Điểm tốt nhất mà Maximizer đã đảm bảo được. Khởi tạo $= -\infty$.
- Beta (β): Điểm tốt nhất mà Minimizer đã đảm bảo được. Khởi tạo $= +\infty$.
- Cắt tia: Khi $\beta \leq \alpha$, dừng duyệt nhánh hiện tại vì kết quả không thể tốt hơn.

Cụ thể trong hệ thống cài đặt hai phiên bản của thuật toán alpha-beta. Điểm khác nhau là phiên bản *alpha_beta_v2* được tối ưu mạnh hơn nhờ sử dụng Transposition Table để lưu lại các trạng thái bàn cờ đã tính toán, giúp tránh việc đánh giá lặp lại. Ngoài ra, phiên bản này còn áp dụng Move Ordering và Beam Search để ưu tiên xét các nước đi tốt nhất và giới hạn số lượng nước đi cần duyệt. Trong khi đó, phiên bản alphabeta đơn giản hơn, chỉ sử dụng cắt tia Alpha-Beta cơ bản kết hợp với danh sách nước đi đã được sắp xếp thông qua *get_ordered_moves()*.

Mã giả thuật toán:

```
def ALPHABETA(board, depth, alpha, beta, is_maximizing:
    NẾU depth = 0 hoặc is_terminal_node(board)
        TRẢ VỀ evaluate_board(board)
    potential_moves ← get_potential_moves(board)
    // Khác biệt:
    // - alpha_beta_v2: dùng cache + beam search
    // - alpha_beta: chỉ dùng move ordering cơ bản
    NẾU is_maximizing
        best_score ←  $-\infty$ 
        CHO MỖI move TRONG potential_moves
            make_move(board, move, BOT_SYMBOL)
            score ← ALPHABETA(board, depth - 1, alpha, beta, FALSE)
            undo_move(board, move)
            best_score ← MAX(best_score, score)
            alpha ← MAX(alpha, best_score)
            NẾU beta ≤ alpha
                BREAK // Cắt tia
        KẾT THÚC CHO
    TRẢ VỀ best_score
```

```

    NGƯỢC LẠI
        best_score ← +∞
        CHO MỖI move TRONG potential_moves
            make_move(board, move, HUMAN_SYMBOL)
score ← ALPHABETA(board, depth - 1, alpha, beta, TRUE)
            undo_move(board, move)
            best_score ← MIN(best_score, score)
            beta ← MIN(beta, best_score)
            NẾU beta ≤ alpha
                BREAK // Cắt tỉa
    KẾT THÚC CHO
    TRẢ VỀ best_score

```

3.2.1 Alpha-beta cơ bản:

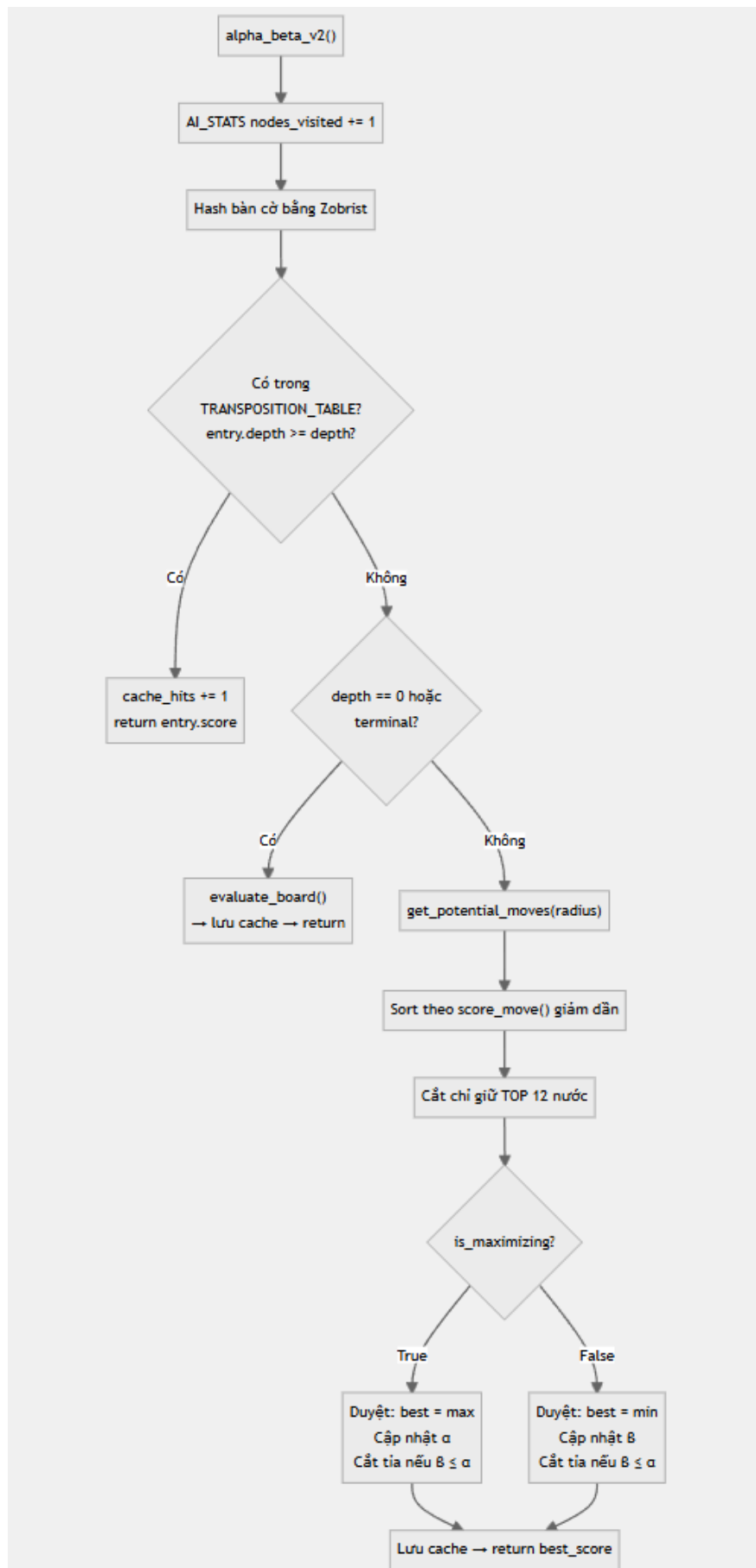
- Trong phiên bản đầu của thuật toán hệ thống bỏ qua một số cách cải tiến như:

Thuộc tính	Giá trị
Transposition Table	Không có — mỗi lượt tính lại từ đầu
Move Ordering	<i>get_ordered_moves()</i> — dùng <i>quick_score()</i> để sắp xếp
Beam Search	Không giới hạn số nhánh
Cắt tỉa	Alpha-Beta cutoff khi $\beta \leq \alpha$

3.2.2. Alpha-Beta Pruning (V2)

Nguyên lý tương tự như phiên bản trước tuy nhiên có một số cải tiến như:

- Transposition Table — Cache kết quả tính toán bằng Zobrist Hash
- Move Ordering nâng cao — Dùng *score_move()* thay vì *quick_score()*
- Beam Search — Giới hạn tối đa 12 nước đi mỗi tầng



Chi tiết các tối ưu:

Transposition Table + Zobrist Hashing

Ưu điểm	Nhược điểm
Tránh tính lại trạng thái trùng lặp	Hash $O(n^2)$ mỗi lần thay vì incremental XOR
Tra cứu dict $O(1)$	Không phân loại EXACT/LOWER/UPPER bound

Move Ordering: *score_move()* vs *quick_score()*

	<i>quick_score()</i> (V1)	<i>score_move()</i> (V2)
Tầm nhìn	Đếm quân liên tiếp sát bên	Quét xa 4 ô mỗi hướng
Phòng thủ	Có nhưng đơn giản	Tính cả tấn công + phòng thủ riêng biệt
Trọng số	Cố định theo mẫu	Phân biệt quân mình/đối thủ (100000 vs 80000)
Vị trí trung tâm	Không xét	Ưu tiên nước gần tâm bàn cờ

Beam Search (Giới hạn độ rộng)

- Ưu điểm: Giảm branching factor từ ~40-60 xuống 12 → cây nhỏ hơn hàng trăm lần
- Nhược điểm: Có thể bỏ lỡ nước đi bất ngờ nằm ngoài top 12

Bảng so sánh các phiên bản thuật toán Caro AI

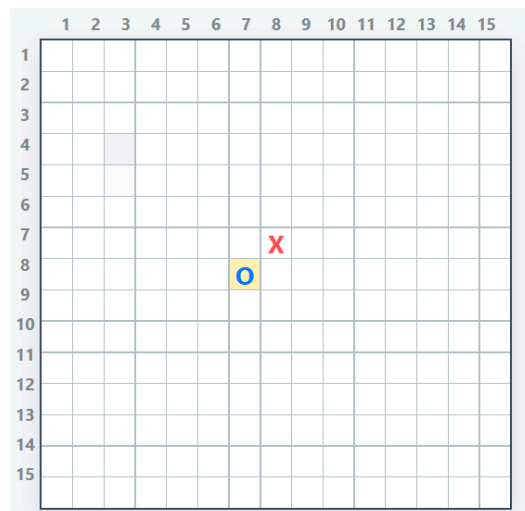
Tiêu chí	Minimax	Alpha-Beta V1	Alpha-Beta V2
Độ sâu mặc định	2	4	Tùy chỉnh
Cắt tỉa Alpha-Beta	✗	✓	✓
Transposition Table	✓	✗	✓
Zobrist Hashing	✓	✗	✓
Move Ordering	<i>score_move()</i>	<i>quick_score()</i>	<i>score_move()</i>
Beam Search	✗ (comment)	✗	✓ (top 12)
Nước đi tăng góc	Tất cả	Tất cả	Top 15

Tiêu chí	Minimax	Alpha-Beta V1	Alpha-Beta V2
Hàm sinh nước đi	get_potential_moves	get_ordered_moves	get_potential_moves
Độ phức tạp	$O(b^d)$	$O(b^{d/2})$ tối ưu	$O(12^d)$ với cache
Tốc độ (Depth = 4)	Chậm nhất	Trung bình	Nhanh nhất
Độ mạnh	Yếu	Trung bình	Mạnh nhất

4. KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH

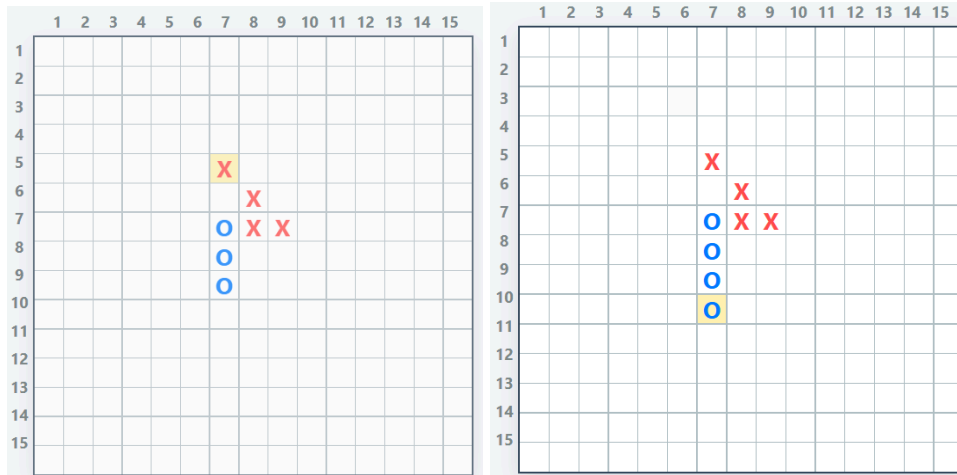
4.1. Thiết kế các trạng thái thử nghiệm

- Trạng thái 1: Đầu ván (bàn cờ trống hoặc ít quân).



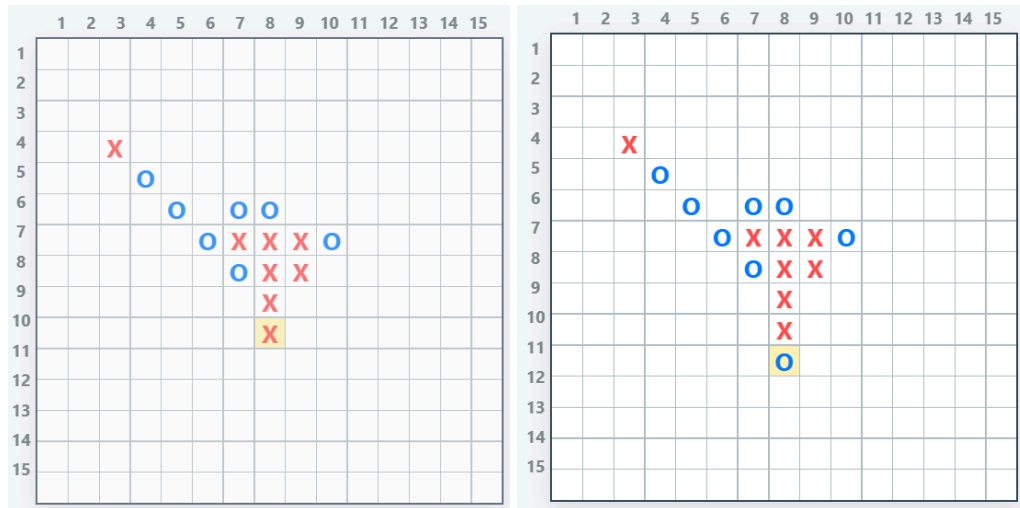
Hình minh họa nước đi của máy (minimax, depth = 2, radius = 1)

- Trạng thái 2: Máy có cơ hội thắng ngay (có chuỗi 3 quân).



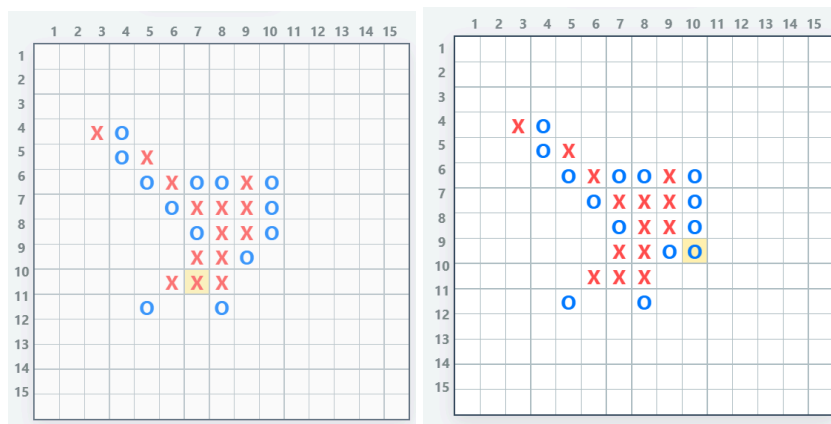
Hình minh họa nước đi của máy (minimax, $depth = 2$, $radius = 1$)

- Trạng thái 3: Người chơi sắp thắng (cần máy chặn).



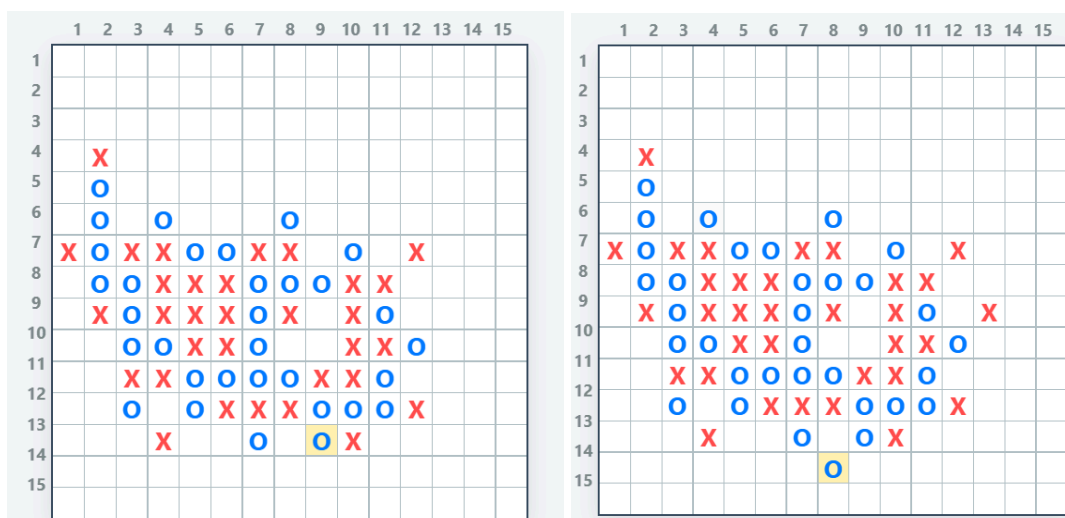
Hình minh họa nước đi của máy (minimax, $depth = 2$, $radius = 1$)

- Trạng thái 4: Giai đoạn giữa ván (phức tạp, nhiều lựa chọn).



Hình minh họa nước đi của máy (minimax, $depth = 2$, $radius = 1$)

- Trạng thái 5: Trạng thái đối kháng mạnh (cả hai bên đều có chuỗi quân).



Hình minh họa nước đi của máy (alpha-beta_v2, depth = 3, radius = 1)

4.2. Bảng kết quả so sánh

Bảng so sánh kết quả với độ sâu depth = 1

Thuật toán	Bàn cờ đã đi 1 nước		Bàn cờ đã đi 10 nước		Bàn cờ đã đi 25 nước	
	Thời gian chạy	Trạng thái đã xét	Thời gian chạy	Trạng thái Đã xét	Thời gian chạy	Trạng thái Đã xét
Minimax với r = 1	0.0309	96	0.2009	593	0.9205	2515
Minimax với r = 2	0.2746	840	1.0976	3231	3.1783	8595
Alpha-beta với r=1	0.0028	8	0.0081	24	0.0144	38
Alpha-beta với r=2	0.0129	24	0.0267	55	0.0367	88
Alpha-beta_v2 với r = 1	0.0067	27	0.0157	42	0.0267	58
Alpha-beta_v2 với r = 2	0.0158	43	0.0202	52	0.0176	46

Bảng so sánh kết quả với độ sâu depth = 2

Thuật toán	Bàn cờ đã đi 1 nước	Bàn cờ đã đi 10 nước	Bàn cờ đã đi 25 nước
------------	---------------------	----------------------	----------------------

	Thời gian chạy	Trạng thái đã xét	Thời gian chạy	Trạng thái Đã xét	Thời gian chạy	Trạng thái Đã xét
Minimax với $r = 1$	1.37	1296	26.51	32854	51.32	71399
Minimax với $r = 2$	35.9019	35800	>300s	> 100000s	>300s	> 100000s
Alpha-beta với $r=1$	0.0314	27	0.2313	167	0.3265	197
Alpha-beta với $r=2$	0.1030	86	0.2644	189	0.2593	157
Alpha-beta_v2 với $r = 1$	0.2891	260	0.5087	394	0.4473	314
Alpha-beta_v2 với $r = 2$	0.4228	448	0.3756	286	0.3918	258

Bảng so sánh kết quả với độ sâu depth = 3

Thuật toán	Bàn cờ đã đi 1 nước		Bàn cờ đã đi 10 nước		Bàn cờ đã đi 25 nước	
	Thời gian chạy	Trạng thái đã xét	Thời gian chạy	Trạng thái Đã xét	Thời gian chạy	Trạng thái Đã xét
Minimax với $r = 1$	17	17776	173.6096	254253	> 200	> 300000
Minimax với $r = 2$	1321.5796	1498051	>300	>100000	> 300	>100000
Alpha-beta với $r=1$	0.1773	140	1.8960	1330	2.5261	1551
Alpha-beta với $r=2$	2.2611	270	6.6642	4786	9.0642	5835
Alpha-beta_v2 với $r = 1$	0.2288	47	0.3313	119	0.4154	1359
Alpha-beta_v2 với $r = 2$	0.2446	1039	0.1668	731	0.1208	387

4.3. Nhận xét và đánh giá

Nhìn chung, thuật toán minimax có thời gian xử lý khá chậm do phải duyệt qua gần như toàn bộ các nước đi có thể xảy ra. Khi trạng thái bàn cờ dần được lấp đầy, số lượng nước đi tiềm năng cũng tăng mạnh, dẫn đến số trạng thái cần đánh giá tăng theo cấp số nhân. Đặc biệt, khi tăng độ sâu tìm kiếm và phạm vi xét nước đi tăng theo cấp số mũ của độ sâu dẫn tới thời gian tính toán trở nên rất lớn, gần như không khả thi trong thực tế. Chỉ với một vài nước đi tiếp theo, số trạng thái cần xét đã có thể lên đến hàng trăm nghìn và tiếp tục tăng mạnh ở các lượt sau, khiến trải nghiệm chơi bị gián đoạn đáng kể.

Đối với thuật toán alpha-beta, hạn chế trên được cải thiện rõ rệt. Điểm nổi bật của thuật toán này là khả năng cắt tỉa các nhánh không cần thiết trong quá trình tìm kiếm, từ đó giảm đáng kể số trạng thái phải xét. Bên cạnh đó, cơ chế sắp xếp nước đi giúp các nước đi có điểm đánh giá cao được ưu tiên xét trước, làm tăng hiệu quả cắt tỉa và cải thiện tốc độ xử lý. Nhờ vậy, dù số lượng nước đi trên bàn cờ tăng lên, thời gian chờ và số trạng thái cần xét chỉ tăng không đáng kể. Hiệu năng chủ yếu bị ảnh hưởng khi tăng độ sâu và độ rộng tìm kiếm, tuy nhiên ở các mức đã thử nghiệm, thuật toán vẫn đáp ứng tốt yêu cầu thực tế.

Về khả năng chơi của máy, việc kết hợp hai thuật toán trên cùng cơ chế *score_move* giúp quá trình đánh giá trạng thái trở nên hiệu quả và thông minh hơn. Máy có khả năng lựa chọn các nước đi hợp lý, khó đoán và có tính chiến thuật cao hơn so với các phương pháp cơ bản. Mức độ thông minh của máy tăng dần theo độ sâu tìm kiếm, tuy nhiên vẫn phụ thuộc nhiều vào chất lượng của hàm đánh giá trạng thái. Trong phạm vi bài tập này, chương trình có thể đánh bại những người mới chơi và tạo được độ khó phù hợp với những người chơi ở mức trung bình hoặc không thường xuyên chơi cờ. Điều này giúp trò chơi duy trì được tính thử thách và tránh cảm giác quá dễ đối với người chơi.

5. TÀI LIỆU THAM KHẢO

Link 1: <https://github.com/husus/gomokuAI-py>

Link 2: https://github.com/MonHauVD/Caro_AI